

Vector stores in the lab: MongoDB Atlas / Voyage-style embeddings

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 11 of 12 · English notes

The hands-on RAG service used a remote **MongoDB Atlas** vector capability and a **Voyage**-style embedding client (spoken in the room as “OH / OHEI”). That pairing is a **class demo**, not the only RAG architecture, and not an Anthropic-owned embedding model. Claude generates text; something else must turn chunks into vectors.

Why the lab is not “the Claude vector database”

Vijay was explicit: unlike some other model labs, **Claude does not ship a first-party embedding model** in this course’s telling. OpenAI- and Amazon-shaped stacks often do. For CCD Foundation you must be able to say: pick an embedding provider, pick a vector store, keep the retrieve-augment-generate loop.

MongoDB+Voyage was what fitted a free cloud database and a client that could both embed and talk to that store.

DO NOT MEMORISE ONE VENDOR AS THE EXAM ANSWER

If a scenario says “Postgres pgvector,” “Chroma,” “FAISS (Meta),” “OpenSearch,” or “Bedrock knowledge base,” that is still RAG. The lab keys and Atlas UI in the PDF folder are homework, not a unique architecture.

What was installed and configured

- Anthropic client for generation after retrieval.
- A Voyage/Mongo vector client, authenticated with an API key stored as something like `VOYAGE_API_KEY` in `.env` (follow the cohort PDF; never commit keys).
- `load_dotenv` so the notebook process can open a remote Atlas-style endpoint.
- A wrapper class (spoken as “vector index”) that knew how to **insert** documents, store 1024-dimension vectors in the demo, and **search** with cosine distance / cosine similarity.

MongoDB here is not “just a document store.” The course used the vendor’s **vector** product: same company brand, different access pattern (k-NN style search on embeddings).

Embeddings in the demo

Sample text went into embed; the notebook printed the first five numbers of a **1024-dimension** array. Those numbers are not meaning you can read; they are coordinates. After all chunks of `report.md` were embedded, the store held on the order of **15 records** in that run — a toy corpus so you could see insert + query, not a production index.

Query path

1. User asks, for example, to explain Fourier series simply.
2. That query string is embedded with the *same* embedding model used at index time (mixing models breaks geometry).
3. Cosine search returns k chunk texts (the lab configured “how many records to fetch”).
4. Those strings are concatenated onto the user question and sent to Claude via the Anthropic API.

Other architectures the trainer named on purpose

Approach	When he mentioned it	What you still must do
MongoDB Atlas + Voyage embeddings	Primary notebook	Chunk, embed, cosine (or configured metric), augment
Amazon Bedrock knowledge base + S3	No-code path: bucket, auto-sync, embedding and chunking in the console, then boto3	Still retrieve then generate; auth via AWS CLI or keys
OpenSearch	Enterprise vector retrieval alternative	Same three letters
Chroma / FAISS, in-memory or local disk	Local tests; LocalStack was suggested instead of paid AWS	You still need an embedding model — Voyage was reused even locally in his answer
Postgres vector extensions	Named in the 24 August RAG teaser	SQL-shaped store, same math

Operational notes from Q&A

- Index-once versus search-on-the-fly: the vector DB is populated as a batch (or a sync pipeline); each user question is a search, not a full rescan of raw Markdown unless you designed it that way.
- Cache embeddings or search results if latency and bill matter.
- Filter with metadata when faculties or tenants must not share hits.
- Similarity defaults belong to `similarity_search`-style APIs; pass parameters if you need a different k or threshold.

Security for UST delivery

A free Atlas cluster and a Voyage key are fine for the medical-research sample file. They are not fine for client PHI/PII. RAG concentrates exactly the documents you were not supposed to paste into Chat. Host the vector store in the approved cloud, encrypt, and keep keys in a secret manager — the notebook `.env` is a teaching crutch.

Official documentation

- **Messages API**

<https://docs.claude.com/en/api/messages>

Where the augmented prompt is sent after search.

- **API overview**

<https://docs.claude.com/en/api/overview>

Auth and clients used beside the vector vendor SDK.

- **PDF support**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Alternative: send documents to Claude without your own vector DB when size allows.

- **Citations**

<https://docs.claude.com/en/docs/build-with-claude/citations>

Ground generation in retrieved or attached text.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Key hygiene when many people share an Anthropic org.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.